智能在线编程练习平台软件设计说明书 (SDD)

编制人: 孙赫

修订日期: 2026 年 04 月 22 日

版本号: V0.2

修订历史

修订人	修订日期	变更原因
冯朝航	2026.04.08	文档首次编制,完成全模块架构与设计说明 [cite: 353]
孙赫	2026.04.22	对齐需求分析文档,优化大纲结构与内容完整性 [cite: 353]

1 概括

1.1 文档目的与受众

本文档是《智能在线编程练习平台需求分析文档》的配套技术基准,完整定义平台的架构设计、模块划分、核心逻辑、接口规范、数据结构与部署方案,面向研发人员、测试人员、项目管理人员、指导老师与评审人员。核心作用:

1. 统一前后端开发、集成测试与质量验收的技术规范,明确模块边界与交互规则

2. 作为项目进度管控、需求变更管理与后续迭代维护的核心技术依据 [cite: 358]
3. 100%覆盖需求文档的功能与非功能要求,保障设计方案可落地、可验证、可交付 [cite: 359]
4. 明确核心技术难点的解决方案,提前识别并规避项目风险 [cite: 360]

1.2 项目定位与范围 [cite: 361]

1.2.1 系统定位 [cite: 362]

基于 SpringBoot 框架开发的 Web 端智能编程练习平台,采用传统 OJ 判题引擎+AI 大模型双引擎模式,打造"官方固定题库+AI 个性化生成题库"双核心体系,为在校学生、算法爱好者、求职备考人群提供一站式算法学习解决方案,同时为管理员提供全流程平台运营管理能力。 [cite: 363]

1.2.2 包含范围 [cite: 364]

- 产品形态: 普通用户刷题 Web 端、管理员后台管理 Web 端 [cite: 365]
- 核心功能模块: 用户管理、题库管理、在线编程与自动判分、AI 智能辅助、数据统计、后台管理 [cite: 366]
- 底层支撑能力: 多语言在线代码编辑器、C++自研 OJ 评测引擎、Docker 安全沙箱、AI 大模型接口对接 [cite: 367]

- 交付内容：平台全量源代码、集成测试报告、性能与安全测试报告、部署手册、全套项目文档

1.2.3 不包含范围

移动端 APP/小程序、多人在线编程竞赛、实时对战、直播教学、付费会员与在线支付、多服务器分布式集群部署、用户社区与评论互动功能。[cite: 370]

1.3 核心术语与缩略语

术语	完整名称	核心定义
SDD	软件设计说明书	描述软件设计目标、架构方案与实现规范的核心技术文档
OJ	在线判题系统	实现用户代码在线编译隔离运行、自动判分的核心引擎模块
CSC	组件级规格说明	软件工程中对单个功能组件的职责、输入输出、核心逻辑与约束的标准化定义
JWT	JSON Web Token	本项目用于无状态用户身份认证的令牌技术
RBAC	基于角色的访问控制	后台权限管理核心模型,通过角色实现用户与权限的解耦
Docker	容器化引擎	用于构建代码运行安全沙箱的容器化技术,实现运行环境完全隔离

1.4 涉众核心诉求

涉众角色	核心关注点
普通用户	代码编辑器无卡顿、判分结果 100%准确且反馈及时、AI 功能可用且内容准确、个人数据安全、核心刷题流程≤3 步
平台管理员	权限管控严格(越权操作 100%拦截)、AI 生成内容可审核、平台运行状态可监控、运营数据可视化
开发人员	架构分层清晰、模块低耦合高内聚、接口规范统一、技术选型匹配团队能力、预留扩展空间
测试人员	功能模块边界明确、异常处理逻辑清晰、性能指标可量化、核心流程可追溯测试
项目负责人	需求 100%覆盖、技术风险可控、工作量匹配 12 周开发周期、符合毕业设计评审要求

1.5 关键设计决策 [cite: 377]

决策编号	核心决策内容	决策依据与核心理由
D-001	前后端分离+B/S 五层单向依赖架构	支持前后端并行开发,职责清晰可维护,避免微服务过度设计与传统
D-002	Docker 容器化沙箱实现代码运行隔离	隔离性强、启动快、资源占用低,杜绝恶意代码攻击,优于虚拟机与
D-003	异步任务队列+WebSocket 实时推送判分流程	削峰填谷支撑高并发,实时性远优于同步等待与轮询方案,提升用户
D-004	AI 智能辅助模块与核心业务完全解耦	AI 服务故障不影响注册刷题、判分等核心功能,提升系统可用性与
D-005	单服务器集中式部署架构	符合项目范围要求,成本低、运维简单,预留后续分布式扩展空间 [
D-006	数据统计采用离线定时任务方案	避免实时计算占用业务高峰期资源,每日更新完全满足学习与运营

1.6 UI 设计规划与需求跟踪表（RTM）【优化补充项】

为确保前后端交互界面的落地及需求的无缝衔接，补充界面规划与跟踪体系：

1.6.1 核心 UI 界面流转规划

- **普通用户端：** 登录/注册页 -> 题库大厅（支持多维度条件筛选） -> 在线刷题工作台（左侧显示题目描述，右侧嵌入 Monaco 编辑器，底部展示终端执行输出） -> 个人中心（错题本、提交记录轨迹）。
- **管理员端：** 仪表盘首页（Echarts 数据概览） -> 题目管理控制台（CRUD） -> AI 内容审核池（提供违规标注与一键下架功能）。

1.6.2 需求跟踪表（RTM）示例

需求编号	需求名称	对应设计模块

REQ-01	用户在线编写代码	在线编程与自动判分模块
REQ-02	系统自动收录错题	用户管理模块
REQ-03	AI 个性化代码点评	AI 智能辅助组件

2 构架设计/总体设计

2.1 整体分层架构

采用经典五层单向依赖架构,无跨层/反向依赖,实现职责分离与层级解耦:

表现层: 用户端 Web | 管理员后台 | 代码编辑器 | 可视化组件

↓

接入层: Nginx 反向代理 | 限流 | 跨域 | HTTPS | 静态资源服务

↓

应用服务层: SpringBoot 核心 | 六大业务模块 | 权限/事务

基础设施层: OJ 引擎 | Docker 沙箱 | AI/邮件客户端 | 缓存/DAO [cite: 390]

数据存储层: MySQL 数据库 | 本地文件存储 | Redis 缓存 [cite: 391]

2.1.1 优化分层架构以体现项目特点 【优化补充项】

为了凸显本项目“判题引擎”和“AI 大模型”双核心的技术难点，对上述通用的“基础设施层”进行拆分解耦：

- **表现层：** Vue 刷题端、管理端、Monaco 代码编辑器组件。

- **应用服务层：** SpringBoot 业务逻辑（题目调度、用户鉴权等）。
- **核心引擎层（项目特色）：** C++ 自研 OJ 调度器、大模型 Prompt 编排与解析引擎。
- **通用设施层：** Nginx、Docker 容器守护进程、Redis。
- **数据存储层：** MySQL 关系型数据库。

2.2 核心模块组成 [cite: 392]

系统分为 6 大独立功能模块,通过标准化接口交互,模块边界清晰、职责单一: [cite: 393]

模块名称	核心职责
用户管理模块	用户全生命周期管理、身份认证、个人数据(提交记录/收藏/错题本)管理 [cite: 394]
题库管理模块	题目、分类、测试用例、官方题解的全生命周期管理与前端展示能力 [cite: 394]
在线编程与自动判分模块	在线代码编辑、代码提交预处理、OJ 判分引擎、沙箱管理、结果实时推送 [cite: 394]
AI 智能辅助模块	AI 代码评价、智能题解、个性化题目生成、编程答疑与内容合规校验 [cite: 394]
数据统计模块	个人学习数据与平台运营数据的离线统计、可视化展示与学习建议生成 [cite: 394]
后台管理模块	用户权限管理、AI 内容审核、平台配置、运行状态监控与操作日志追溯 [cite: 395]

2.3 部署架构 [cite: 396]

采用单服务器集中式部署,所有组件运行于同一台 Linux 云服务器,无跨服务器节点交互: [cite: 397]

软件组件	部署方式	核心作用
Nginx 1.24	系统服务安装	静态资源服务、请求转发、HTTPS 加密、接口限流 [cite

前端静态资源	Nginx 目录部署	用户端与管理员端页面文件 [cite: 398]
SpringBoot 2.7.x	Jar 包+系统服务管理	核心业务逻辑与 API 接口提供 [cite: 398]
MySQL 5.7	系统服务安装	结构化业务数据存储 [cite: 398]
Redis 6.2	系统服务安装	缓存、限流、会话管理 [cite: 398]
Docker 24.0	系统服务安装	代码运行安全沙箱环境管理 [cite: 398]

2.3.1 进程级映射架构对应 【优化补充项】

上述单服务器部署与分层架构的一一对应关系及通信逻辑如下：

- **Nginx (80/443 端口)：** 承载表现层（分发 Vue 静态文件），并作为应用服务层的反向代理，解决前后端分离的跨域问题。
- **SpringBoot (8080 端口)：** 承载应用服务层的所有 6 个核心业务模块（C-USER-001 至 C-ADMIN-001），通过 RESTful API 对前端暴露服务。
- **Docker Daemon：** 承载核心引擎层的沙箱运行环境。SpringBoot 通过 Docker API (Unix Socket 或 TCP) 与其进行内部通信，动态创建和销毁隔离容器。

2.4 数据架构 [cite: 399]

2.4.1 数据库设计原则 [cite: 400]

- 严格遵循数据库三大范式,减少数据冗余,核心数据采用逻辑删除而非物理删除 [cite: 401]

- 针对高频查询字段建立合适索引,避免全表扫描,保障查询性能 [cite: 402]
- 敏感字段(密码、邮箱)加密存储,用户隐私数据脱敏展示 [cite: 403]
- 统一命名规范:表名/字段名采用下划线命名法,所有表包含主键、创建时间、更新时间字段 [cite: 404]

2.4.2 核心数据表设计 [cite: 405]

表名	核心用途	关键索引设计
sys_user	用户基础信息存储	用户名唯一索引、邮箱唯一索引、状态索引 [
sys_role	角色信息存储	角色标识唯一索引 [cite: 406]
sys_user_role	用户-角色关联存储	user_id+role_id 联合唯一索引 [cite: 406]
problem_info	题目基础信息存储	标题、难度、状态、创建时间索引 [cite: 406
problem_test_case	题目测试用例存储	题目 ID+是否公开联合索引 [cite: 406]
problem_solution	题目官方题解存储	题目 ID 唯一索引、状态索引 [cite: 406]
code_submission	代码提交记录存储	用户 ID、题目 ID、提交状态、创建时间索引
user_error_book	用户错题本存储	用户 ID、题目 ID、掌握状态索引 [cite: 406]
ai_generated_content	AI 生成内容存储	用户 ID、内容类型、状态索引 [cite: 406]
sys_operation_log	系统操作日志存储	用户 ID、操作类型、创建时间索引 [cite: 40

2.4.4 核心数据表关联关系与 SQL 实现示例 【优化补充项】

为了明确体现核心数据表之间的关系，以下提供“用户-提交记录-错题本”核心链路的 `user_error_book`（错题本）建表 SQL 示例。该表通过明确的逻辑关联连接了系统三大核心数据：

```
-- 用户错题本表设计示例
```

```sql
CREATE TABLE `user_error_book` (

    `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '主键',

    `user_id` BIGINT NOT NULL COMMENT '关联 sys_user 表主键',

    `problem_id` BIGINT NOT NULL COMMENT '关联 problem_info 表主键',

    `last_submit_id` BIGINT COMMENT '最近一次错误提交记录，关联 code_submission 表',

    `mastery_status` TINYINT DEFAULT 0 COMMENT '掌握状态：0-未掌握，1-已掌握',

    `create_time` DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',

    `update_time` DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',

    PRIMARY KEY (`id`),

    -- 核心联合索引，保证单用户对单题目只有一条错题汇总记录

    UNIQUE KEY `uk_user_problem` (`user_id`, `problem_id`)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户错题本';
```

2.4.3 数据生命周期管理 [cite: 407]

- 用户账号注销后,按合规要求删除个人敏感数据,提交记录与操作日志匿名化保留 [cite: 408]
- 代码提交记录、操作日志保留不少于 1 年 [cite: 409]
- 数据库每日凌晨执行全量自动备份,备份文件保留不少于 30 天 [cite: 410]
- 系统日志、临时文件自动清理,仅保留最近 7 天数据 [cite: 411]

2.5 核心状态机设计 [cite: 412]

2.5.1 代码提交状态机 [cite: 413]

- 核心状态:待提交预处理中→待判分→判分中→[判分通过/判分不通过/编译失败/运行超时/内存超限/系统异常] [cite: 414, 415]
- 初始状态:待提交 [cite: 416]
- 流转规则:用户提交代码→预处理校验通过进入待判分→判分引擎接管进入判分中→根据执行结果进入终态 [cite: 417, 418]
- 约束:所有终态不可逆向流转,用户重新提交生成新的提交记录,不修改原有记录 [cite: 419]

2.5.2 题目状态机 [cite: 420]

- 核心状态:草稿→待审核→已上线→已下架→已删除 [cite: 421]
- 初始状态:草稿 [cite: 422]
- 流转规则:管理员新增题目→草稿→提交审核进入待审核→审核通过进入已上线→需修改/下架进入已下架→可重新编辑或删除 [cite: 423, 424]
- 约束:仅已上线状态的题目可被普通用户查看与检索 [cite: 425]

3 功能设计(软件工程 CSC) [cite: 426]

3.1 用户管理组件 [cite: 427]

- 组件 ID: C-USER-001 [cite: 428]
- 组件职责:提供用户身份认证与全生命周期数据管理能力,保障账号安全 [cite: 429]

- 核心输入:注册/登录请求、个人信息修改请求、收藏/错题操作请求、账号注销请求 [cite: 430]
- 核心输出: JWT 令牌、用户信息、提交记录列表、收藏/错题数据、操作结果 [cite: 431]
- 核心功能: [cite: 432]

    1. 账号注册:用户名/邮箱唯一性校验、邮箱验证码验证、密码 BCrypt 不可逆加密存储 [cite: 433]
    2. 账号登录:支持用户名/邮箱两种方式,连续 5 次失败临时锁定 15 分钟,登录状态有效期 7 天 [cite: 434, 435]
    3. 个人信息管理:修改昵称、头像、绑定邮箱,头像限制 JPG/PNG 格式且<2MB [cite: 436, 437]
    4. 密码管理:原密码校验修改、邮箱验证码重置,修改后原有登录状态自动失效 [cite: 438]
    5. 提交记录管理:分页展示本人提交记录,支持按题目、时间、状态、语言筛选 [cite: 439]
    6. 收藏与错题本:手动收藏题目,系统自动收录未通过题目至错题本,支持标记掌握状态 [cite: 440]
    7. 账号注销:身份校验后合规删除个人数据,注销后不可恢复 [cite: 441]

- 约束条件:普通用户仅可访问本人数据;敏感操作全流程日志记录,可溯源 [cite: 442]

3.2 题库管理组件 [cite: 443]

- 组件 ID: C-PROBLEM-001 [cite: 444]
- 组件职责:管理平台核心内容资产,为前端提供题目检索与展示能力 [cite: 445]
- 核心输入:题目增删改查请求、测试用例导入/导出请求、分类管理请求 [cite: 446]
- 核心输出:题目列表、题目详情、测试用例数据、官方题解、操作结果 [cite: 447]
- 核心功能: [cite: 448]

    1. 题目分类管理:新增、编辑、删除、排序、启用/禁用分类,题目与分类多对多绑定 [cite: 449]
    2. 题目基础信息管理:新增、编辑、上架、下架、删除题目,管理标题、描述、难度、时间/内存限制等 [cite: 450]
    3. 测试用例管理:管理公开样例与隐藏用例,支持单条操作与批量导入/导出,隐藏用例对普通用户不可见 [cite: 451]
    4. 官方题解管理:编辑、发布、下架官方标准解析与参考代码,审核通过后方可展示 [cite: 452]
    5. 题目检索与展示:分页展示已上线题目,支持按分类、难度、关键词筛选,按热度/时间排序 [cite: 453]

- 约束条件:已绑定题目的分类不可直接删除;题目修改不回溯历史判分结果;下架题目对普通用户不可见 [cite: 454]

3.3 在线编程与自动判分组件 [cite: 455]

- 组件 ID: C-JUDGE-001 [cite: 456]
- 组件职责:平台核心引擎,实现代码在线运行与自动判分,保障判分准确性与安全性 [cite: 457]
- 核心输入:用户代码、题目 ID、编程语言、测试用例数据、运行限制参数 [cite: 458]
- 核心输出:提交编号、判分进度、最终判分报告、运行耗时/内存数据 [cite: 459]
- 核心功能: [cite: 460]

  1. 在线代码编辑器:基于 Monaco Editor 实现,支持 C++/Java/Python,提供代码高亮、自动缩进、格式化、自动保存草稿 [cite: 461]
  2. 代码预处理:非空校验、行数限制(≤10000 行)、静态恶意代码检测 [cite: 462]
  3. 异步判分调度:生成唯一提交编号,将任务推入 FIFO 队列,单用户每秒最多提交 1 次 [cite: 463]
  4. Docker 沙箱运行:每次运行创建独立容器,单任务限制 1 核 CPU/256M 内存/最长 5 秒运行,禁用外网访问 [cite: 464, 465]
  5. 自动判分逻辑:编译代码→逐条执行测试用例→标准化输出比对(忽略行尾空格/换行符)→汇总结果 [cite: 466]

6. 结果实时推送:通过 WebSocket 推送单条用例执行进度与最终判分结果 [cite: 467]

- 约束条件:容器运行结束立即销毁;判分结果准确率 100%;常规题目(≤10 用例)判分最长不超过 8 秒 [cite: 468]

3.3.1 核心组件设计落地：判分接口规范定义 【优化补充项】

将功能需求转化为具体的代码设计层，指导开发人员落实：

```
// 核心判分调度接口定义示例

public interface JudgeService {

    /**

     * 提交代码进行异步判分

     * @param request 包含用户代码、题目 ID、语言类型

     * @return submitId 提交编号，前端利用此编号开启 WebSocket 监听

     */

    Long submitCode(JudgeSubmitRequest request);



    /**

     * 沙箱运行完毕后回调更新判分结果

     * @param submitId 提交编号

     * @param result 封装了运行耗时、内存占用、状态码(如 AC/WA/TLE 等)的实体

     */

    void updateJudgeResult(Long submitId, JudgeResult result);

}
```

3.4 AI 智能辅助组件 [cite: 469]

- 组件 ID: C-AI-001 [cite: 470]
- 组件职责:对接第三方 AI 大模型,提供编程学习智能辅助能力,保障内容合规 [cite: 471]
- 核心输入:提交记录 ID、题目 ID、用户提问内容、生成参数 [cite: 472]
- 核心输出:代码评价、错题解析、生成题目、答疑内容 [cite: 473]
- 核心功能: [cite: 474]

    1. AI 代码多维评价:从运行效率、代码规范、可读性、健壮性四个维度评价,给出优化建议与完整代码 [cite: 475]
    2. AI 智能题解:生成通用标准化题解,针对错误代码生成个性化错题解析,定位错误原因 [cite: 476]
    3. AI 个性化题目生成:支持自定义算法分类、难度、场景,生成完整题目、测试用例与题解 [cite: 477]
    4. AI 编程答疑:对话式解答编程相关问题,单轮会话最多 10 轮,仅回答编程学习相关内容 [cite: 478, 479]
    5. 内容合规校验:所有 AI 生成内容自动审核,违规内容不展示 [cite: 480]

- 约束条件:单用户每日调用限流(代码评价 50 次/错题解析 30 次/题目生成 20 道); AI 故障不影响核心业务流程 [cite: 481, 482]

3.5 数据统计组件 [cite: 483]

- 组件 ID: C-STAT-001 [cite: 484]
- 组件职责:离线统计学习与运营数据,提供可视化展示与决策支撑 [cite: 485]
- 核心输入:业务数据库全量数据、统计时间维度 [cite: 486]
- 核心输出:个人学习报告、平台运营仪表盘、学习建议 [cite: 487]
- 核心功能: [cite: 488]

    1. 个人学习数据统计:每日凌晨离线统计刷题量、正确率、知识点掌握进度、学习趋势等 [cite: 489]
    2. 平台运营数据统计:统计用户注册量、题库数据、判分成功率、AI 功能使用率等 [cite: 490]
    3. 数据可视化:基于 ECharts 实现图表展示,异常指标高亮提示 [cite: 491]
    4. 学习建议生成:基于用户学习数据给出针对性学习方向建议 [cite: 492]

- 约束条件:普通用户仅可查看本人数据;管理员按权限分级查看运营数据 [cite: 493]

3.6 后台管理组件 [cite: 494]

- 组件 ID: C-ADMIN-001 [cite: 495]
- 组件职责:提供平台运营与管控能力,保障平台安全稳定运行 [cite: 496]

- 核心输入:管理员操作请求、权限校验参数、平台配置参数 [cite: 497]
- 核心输出:用户列表、AI 待审核内容、平台运行状态、操作日志 [cite: 498]
- 核心功能: [cite: 499]

    1. 用户与权限管理:基于 RBAC 模型分配角色与权限,启用/禁用用户账号,查看操作日志 [cite: 500]
    2. AI 内容审核:查看所有 AI 生成内容,完成审核、编辑、纳入官方题库、下架操作 [cite: 501]
    3. 平台配置管理:超级管理员可配置平台基础参数 [cite: 502]
    4. 运行状态监控:查看接口响应、判分引擎状态、AI 接口调用情况,接收异常告警 [cite: 503, 504]

- 约束条件:所有后台操作双重权限校验;全流程日志记录,操作人、时间、IP 可溯源 [cite: 505]

4 接口设计 [cite: 506]

4.1 通用接口规范 [cite: 507]

1. 通信协议:常规业务接口采用 HTTPS RESTful 协议,实时推送接口采用 WebSocket 协议 [cite: 508, 509]
2. 请求方法:查询操作使用 GET,新增操作使用 POST,修改操作使用 PUT,删除操作使用 DELETE [cite: 510]

3. 统一响应格式: [cite: 511]

```
{
  "code": 200, // 200 成功,4xx 客户端错误,5xx 服务端错误
  "msg":"操作成功",
  "data": null // 响应数据,无数据时为 null
}
```

4. [cite: 512-517]

5. 权限规范:除注册、登录、题目浏览等公开接口外,所有接口请求头需携带 `Authorization: Bearer {JWT 令牌}` [cite: 518, 519]
6. 异常规范:所有异常均捕获并返回明确错误码与提示信息,无未捕获异常导致的服务崩溃 [cite: 520]

4.4 第三方服务接口规范 [cite: 594]

1. AI 大模型接口:

   封装统一客户端,支持主备两个渠道切换 [cite: 596]

   超时时间设置为 20 秒,超时重试 3 次,连续失败 5 次触发熔断 [cite: 597]

   所有请求与响应日志记录,便于问题排查 [cite: 598]
2. 邮件验证码接口:

   同一邮箱 1 分钟内最多发送 1 次,1 小时内最多发送 5 次 [cite: 599, 600]

   验证码有效期 15 分钟,使用后立即失效 [cite: 601]

   发送失败记录日志,不影响用户核心操作 [cite: 602]

4.4.1 第三方服务具体技术对接细节 【优化补充项】

- **AI 大模型服务商对接作用：** 优先采用阿里云通义千问 API (如 `qwen-turbo`)，备用渠道为腾讯混元大模型。具体对接过程：当用户触发错题解析时，后端拼接用户代码、报错信息与预设系统指令（Prompt），通过 HTTP POST 请求向 `/v1/chat/completions` 发起调用，获取代码修改意见。
- **邮件发送服务商对接作用：** 采用腾讯企业邮箱 SMTP 服务。后端集成 Spring Boot Starter Mail，利用授权码登录 SMTP 服务器，在注册和找回密码时，异步渲染包含 6 位随机验证码的 HTML 邮件模板并向用户发送。

5 性能指标设计 [cite: 603]

5.4 判分引擎性能指标 [cite: 610]

指标项	要求值
常规题目(≤10 用例)判分耗时	平均 ≤3 秒 [cite:
常规题目最大判分耗时	≤8 秒 [cite: 611]
Docker 容器启动时间	≤500 毫秒 [cite: 6
判分任务队列容量	≥1000 个 [cite: 61
判分结果准确率	100% [cite: 611]

5.5 AI 功能性能指标 [cite: 612]

指标项	要求值
AI 代码评价平均响应	≤10 秒 [cite: 613]
AI 代码评价最大响应	≤15 秒 [cite: 613]

AI 题目生成平均响应	≤15 秒 [cite: 613]
AI 题目生成最大响应	≤20 秒 [cite: 613]
AI 功能调用成功率	≥99% [cite: 613]

5.7 性能指标设计合理性与实现路径说明 【优化补充项】

对于上述关键性能指标的要求，补充论证其设定合理性及工程实现手段：

- **AI 响应指标合理性与来源：**规定“AI 评价平均响应 ≤10 秒”，是基于目前主流大语言模型处理 1000 Token（包含代码上下文和结构化评语生成）的平均网络开销测试得出的合理区间。
- **AI 响应性能实现路径：**采用 **WebSocket 流式输出（Stream）。**大模型接口响应无需等待全量文本生成完毕，而是像“打字机”一样逐字推送到前端，保证用户的“首字响应时间”控制在 2 秒以内，从交互体验上规避等待焦虑。

6 硬件设计 [cite: 617]

6.2 服务器配置要求 [cite: 622]

硬件项	推荐配置要求
CPU	4 核 vCPU [cite: 623]
内存	8GB [cite: 623]
硬盘	200GB SSD [cite: 623]
公网带宽	10Mbps [cite: 624]

6.3 硬件资源管控规则 [cite: 625]

1. 计算资源管控：代码运行沙箱单任务 1 核 CPU、256MB 内存、最长 5 秒运行时间；后端 JVM 堆内存固定为 2GB；数据库 InnoDB 缓冲池大小固定为 1GB。 [cite: 626-629]
2. 存储资源管控：单代码文件≤1MB、≤10000 行；用户头像≤2MB。[cite: 630-632]

6.3.1 对应架构的硬件资源分配清单明细 【优化补充项】

以推荐的 8GB 内存配置为例，服务器上各核心组件与架构设计的资源硬隔离分配如下，确保高并发时各层互不挤兑：

- **应用服务层（SpringBoot JVM）：** 锁定分配 2GB（`-Xms2g -Xmx2g`）。
- **数据存储层（MySQL）：** 限制分配 1.5GB（缓冲池预留 1GB 保障高频查询）。
- **通用设施层（Redis）：** 分配 0.5GB，采用 `allkeys-lru` 策略防止内存溢出。
- **核心引擎层（Docker 沙箱池）：** 预留 3GB。可支撑系统同时并发启动 10-12 个（单次限制 256MB）评测容器进程。
- **系统 OS 及 Nginx：** 保留 1GB 以维持操作系统和代理服务的稳定。

7 其他设计 [cite: 644]

7.1.2 代码运行安全 [cite: 652]

- 采用 Docker 容器构建独立沙箱环境,每次运行创建新容器,运行结束立即销毁 [cite: 653]
- 沙箱严格限制 CPU、内存、运行时长,禁止系统命令执行、root 权限与宿主机目录挂载 [cite: 654]
- 代码提交前进行静态恶意代码检测,检测到危险代码直接禁止提交 [cite: 655]
- 代码运行时全程监控系统调用,异常行为立即终止容器 [cite: 656]

7.2 可靠性设计 [cite: 666]

- 核心解耦设计:AI 智能辅助模块与核心业务流程完全解耦,AI 服务故障不影响用户注册、刷题、判分等核心功能 [cite: 667]
- 异步处理机制:判分、AI 调用、数据统计等耗时操作采用异步任务队列处理,不阻塞主线程 [cite: 668]
- 异常隔离机制:单个用户代码异常不影响其他用户,单个任务失败不导致系统崩溃 [cite: 670, 671]